Programming Languages and Environments (Lecture 14)

LEI - Licenciatura em Engenharia Informática

João Costa Seco (joao.seco@fct.unl.pt)

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

Syllabus

- Module systems
- Modules in OCaml
- Functors in OCaml, Scala and RUST Traits
- Mutability

Module systems

- Name spaces
  - Groups of declarations (usually) related, isolated from other groups through name qualification in a module.
  - It allows the reuse of the same names in different contexts without collisions.
  - Examples: Packages and classes in Java, files/modules in C, structures/modules in OCaml.

C

```c
#include "stack.h"
// int push(int x);

#include "queue.h"
// int push(int x);

/* CONFLICT - duplicate definition */
```

Java

```java
import java.util.Date;
import java.sql.Date;

java.util.Date now     = new java.util.Date();
java.sql.Date  sqlDate = new java.sql.Date(now.getTime());
```

Module systems

- Name spaces
  - Groups of declarations (usually) related, isolated from other groups through name qualification in a module.
  - It allows the reuse of the same names in different contexts without collisions.
  - Examples: Packages and classes in Java, files/modules in C, structures/modules in OCaml.
- Abstraction
  - Allows selectively hiding/revealing information (*information hiding*).
    - Declarations in C “header” files and Java interfaces.
  - Code isolation, better development and maintenance, ownership, etc.

Java

```java
Map<Integer, String> hm = new HashMap<>();
Map<Integer, String> ht = new TreeMap<>();
hm.put(1, "one");
hm.put(2, "two");
hm.put(3, "three");
ht.put(1, "one");
ht.put(2, "two");
ht.put(3, "three");
```

Module systems

- Name spaces
  - Groups of declarations (usually) related, isolated from other groups through name qualification in a module.
  - It allows the reuse of the same names in different contexts without collisions.
  - Examples: Packages and classes in Java, files/modules in C, structures/modules in OCaml.
- Abstraction
  - Allows selectively hiding/revealing information (*information hiding*).
  - Code isolation, better development and maintenance, ownership, etc.
- Code reuse
  - Reuse without copying, modularity (cf. inheritance in Java).

Module systems

- Name spaces
  - Groups of declarations (usually) related, isolated from other groups through name qualification in a module.
  - It allows the reuse of the same names in different contexts without collisions.
  - Examples: Packages and classes in Java, files/modules in C, structures/modules in OCaml.
- Abstraction
  - Allows selectively hiding/revealing information (*information hiding*).
  - Code isolation, better development and maintenance, ownership, etc.
- Code reuse
  - Reuse without copying, modularity (cf. inheritance in Java).
- (in OCaml) Module parametrization, (in Scala, Rust) Traits
  - Functors in OCaml are like functions from modules to modules (cf. traits in Scala or Rust).

Module systems

- Name space
  - Groups of dec
  - It allows the re
  - Examples: Pa
- Abstraction
  - Allows selecti
  - Code isolation
- Code reuse
  - Reuse without
- (in OCaml) M
  - Functors in O

ation in a module.

ml.

```
trait Map[K, V] {
  def put(key: K, value: V): Unit
  def get(key: K): Option[V]
}

class HashMap[K, V] extends Map[K, V] {
  private val data = scala.collection.mutable.HashMap[K, V]()
  def put(key: K, value: V): Unit = data(key) = value
  def get(key: K): Option[V] = data.get(key)
}

class TreeMap[K, V](implicit ord: Ordering[K]) extends Map[K, V] {
  private val data = scala.collection.mutable.TreeMap[K, V]()
  def put(key: K, value: V): Unit = data(key) = value
  def get(key: K): Option[V] = data.get(key)
}

val m1: Map[Int, String] = new HashMap()
m1.put(1, "one")
println(m1.get(1))

val m2: Map[Int, String] = new TreeMap()
m2.put(1, "one")
println(m2.get(1))
```

Module systems

- Na
  - C
  - It
  - E
- Ab
  - A
  - C
- Co
  - R
- (in
  - F

```
trait Printable[K, V] { self: Map[K, V] =>
  def prettyPrint(): Unit = {
    println(s"[${this.getClass.getSimpleName}]")
    println(this.toString)
  }
}

class HashMap[K, V] extends scala.collection.mutable.HashMap[K, V]
    with Printable[K, V]

class TreeMap[K, V](implicit ord: Ordering[K]) extends scala.collection.mutable.TreeMap[K, V]
    with Printable[K, V]

val hm = new HashMap[Int, String]()
hm.put(1, "one")
hm.put(2, "two")

val tm = new TreeMap[Int, String]()
tm.put(1, "one")
tm.put(2, "two")

hm.prettyPrint()
tm.prettyPrint()
```

Module systems

- Name spaces
  - Groups of declarations (usually) r
  - It allows the reuse of the same na
  - Examples: Packages and classes
- Abstraction
  - Allows selectively hiding/revealing
  - Code isolation, better developmer
- Code reuse
  - Reuse without copying, modularity
- (in OCaml) Module parametr
  - Functors in OCaml are like functio

n in a module.

```rust
use std::collections::{HashMap, BTreeMap};
use std::any::type_name;

// A flexible extension to a struct that implements function "entries"
trait Printable {
    fn entries(&self) -> Vec<(i32, String)>;

    fn pretty_print(&self) {
       for (k, v) in self.entries() {
            println!("  {} => {}", k, v);
        }
    }
}

// Extension of HashMap with Printable trait
impl Printable for HashMap<i32, String> {
    fn entries(&self) -> Vec<(i32, String)> {
        self.iter().map(|(k, v)| (*k, v.clone())).collect()
    }
}

// Extension of BTreeMap with Printable trait
impl Printable for BTreeMap<i32, String> {
    fn entries(&self) -> Vec<(i32, String)> {
        self.iter().map(|(k, v)| (*k, v.clone())).collect()
    }
}

fn main() {
    let mut hm = HashMap::new();
    hm.insert(1, "one".to_string());
    hm.insert(2, "two".to_string());

    let mut bt = BTreeMap::new();
    bt.insert(1, "one".to_string());
    bt.insert(2, "two".to_string());

    hm.pretty_print();
    bt.pretty_print();
}
```

Modules in OCaml

Modules in OCaml

- Modules are defined by **structures** (`struct`).
- The types for the modules are **signatures** (`sig`).
- The type definitions by default are **public** (`type`).
- The implementations of names are **private** (`val`).

```
module MyList = struct
  type 'a list = Nil | Cons of 'a * 'a list
  let empty = Nil
  let rec length = function
    | Nil → 0
    | Cons (_, xs) → 1 + length xs
  let insert x xs = Cons (x, xs)
  let head = function
    | Nil → None
    | Cons (x, _) → Some x
  let tail = function
    | Nil → None
    | Cons (_, xs) → Some xs
end
```

[7] ✓ 0.0s

```
module MyList :
  sig
    type 'a list = Nil | Cons of 'a * 'a list
    val empty : 'a list
    val length : 'a list → int
    val insert : 'a → 'a list → 'a list
    val head : 'a list → 'a option
    val tail : 'a list → 'a list option
  end
```

Name spaces

- The names declared in a module can be used in a qualified manner (with the name followed by a dot: **`List.fold_right`**).
- Alternatively, the **`open`** directive can be used to expand the names of the module in the client module.
- The **`Stdlib`** module is always open.

```
module MyStack = struct
  type 'a stack = 'a MyList.list
  let empty = MyList.empty
  let push x xs = MyList.insert x xs
  let pop = MyList.tail
  let top = MyList.head
end
```

[17] ✓ 0.0s

```
module MyStack :
  sig
    type 'a stack = 'a MyList.list
    val empty : 'a MyList.list
    val push : 'a → 'a MyList.list → 'a MyList.list
    val pop : 'a MyList.list → 'a MyList.list option
    val top : 'a MyList.list → 'a option
  end
```

```
module MyStack = struct
  open MyList

  type 'a stack = 'a list
  let empty = empty
  let push x xs = insert x xs
  let pop = tail
  let top = head
end
```

[14] ✓ 0.0s

```
module MyStack :
  sig
    type 'a stack = 'a MyList.list
    val empty : 'a MyList.list
    val push : 'a → 'a MyList.list → 'a MyList.list
    val pop : 'a MyList.list → 'a MyList.list option
    val top : 'a MyList.list → 'a option
  end
```

Name abstraction

- The types of modules also allow hiding the definition of types.
- A signature can have multiple compatible (opaque) implementations.

```
module type Stack =
  sig
    type 'a stack
    val empty : 'a stack
    val push : 'a → 'a stack → 'a stack
    val pop : 'a stack → 'a stack option
    val top : 'a stack → 'a option
  end

module MyStack : Stack

module AnotherStack : Stack
```

```
module type Stack = sig
  type 'a stack
  val empty : 'a stack
  val push : 'a → 'a stack → 'a stack
  val pop : 'a stack → 'a stack option
  val top : 'a stack → 'a option
end

module MyStack:Stack = struct
  type 'a stack = 'a MyList.list
  let empty = MyList.empty
  let push x xs = MyList.insert x xs
  let pop = MyList.tail
  let top = MyList.head
end

module AnotherStack : Stack = struct
  type 'a stack = 'a list
  let empty = []
  let push x xs = x :: xs
  let pop = function
    | [] → None
    | _ :: xs → Some xs
  let top = function
    | [] → None
    | x :: _ → Some x
end
```

Types and names

- The specialization of modules can be done with an adaptation module.

```
module IntStack = (struct
  type stack = int MyStack.stack
  let empty = MyStack.empty
  let push = MyStack.push
  let pop = MyStack.pop
  let top = MyStack.top
end : sig
  type stack
  val empty : stack
  val push : int → stack → stack
  val pop : stack → stack option
  val top : stack → int option
end)
```

[36] ✓ 0.0s

```
module IntStack :
  sig
    type stack
    val empty : stack
    val push : int → stack → stack
    val pop : stack → stack option
    val top : stack → int option
  end
```

Modules and Files

- The organization into separate files separates the structure (`struct`) from the signature (`sig`).
- Files like `MyList.ml`, `MyStack.mli`, and `MyStack.ml` are examples.

s > LAP 2024-12 > MyList.ml > ...

```
type 'a list = Nil | Cons of 'a * 'a list
'a list
let empty = Nil
'a list -> int
let rec length = function
  | Nil → 0
  | Cons (_, xs) → 1 + length xs
'a -> 'a list -> 'a list
let insert x xs = Cons (x, xs)
'a list -> 'a option
let head = function
  | Nil → None
  | Cons (x, _) → Some x
'a list -> 'a list option
let tail = function
  | Nil → None
  | Cons (_, xs) → Some xs
```

> LAP 2024-12 > MyStack.mli > ...

```
type 'a stack
val empty : 'a stack
val push : 'a → 'a stack → 'a stack
val pop : 'a stack → 'a stack option
val top : 'a stack → 'a option
```

s > LAP 2024-12 > MyStack.ml > ...

```
type 'a stack = 'a MyList.list
'a
let empty = MyList.empty
'a -> 'b -> 'c
let push x xs = MyList.insert
'a
let pop = MyList.tail
'a
let top = MyList.head
```

- Promotes separate compilation

Modules and Functors (module-to-module functions)

```
module type X = sig
  val x : int
end

module IncX (M : X) = struct
  let x = M.x + 1
end
```

[23] ✓ 0.0s

```
module type X = sig val x : int end

module IncX : functor (M : X) → sig val x : int end
```

Modules and Functors (module-to-module functions)

```
module type X = sig
  type t
end

module Stack = struct
  module Make (M : X) = struct
    type stack = M.t list
    let empty = []
    let push x xs = x :: xs
    let pop = function
      | [] → None
      | _ :: xs → Some xs
    let top = function
      | [] → None
      | x :: _ → Some x
  end
end

module IntStack = Stack.Make (struct type t = int end)

let s = IntStack.empty
let _ = assert (IntStack.top s = None)
let _ = assert (IntStack.top (IntStack.push 1 s) = Some (1))
let _ = assert (IntStack.pop (IntStack.push 1 s) = Some (IntStack.empty))
```

[71] ✓ 0.0s

Modules and Functors (module-to-module functions)

```
module type OrderedType = sig (* from the standard library *)
  type t
  val compare : t -> t -> int
end

module Pair = struct
  type t = int * string
  let compare (x1, y1) (x2, y2) =
    if x1 < x2 then -1
    else if x1 > x2 then 1
    else if y1 < y2 then -1
    else if y1 > y2 then 1
    else 0
end

module PairSet = Set.Make(Pair)

let s = PairSet.empty
let s = PairSet.add (1, "one") s
let s = PairSet.add (2, "two") s
let s = PairSet.add (3, "three") s
let s = PairSet.add (4, "four") s
let _ = assert (PairSet.mem (2, "two") s)
```

Modules and Functors (module-to-module functions)

```
module type OrderedType = sig (* from the standard library *)
  type t
  val compare : t -> t -> int
end

module Pair = struct
  type t = int * string
  let compare (x1, y1) (x2, y2) =
    if x1 < x2 then -1
    else if x1 > x2 then 1
    else if y1 < y2 then -1
    else if y1 > y2 then 1
    else 0
end

module PairSet = Set.Make(Pair)

let s = PairSet.empty
let s = PairSet.add (1, "one") s
let s = PairSet.add (2, "two") s
let s = PairSet.add (3, "three") s
let s = PairSet.add (4, "four") s
let _ = assert (PairSet.mem (2, "two") s)
```

Modules and Functors (module-to-module functions)

```
module StringMap = Map.Make(String) (* from the standard library *)

let m = StringMap.empty
let m = StringMap.add "joao" 30 m
let m = StringMap.add "maria"   25 m
let m = StringMap.add "carlos" 35 m

let _ = assert (StringMap.find "joao" m = 30)
let _ = assert (StringMap.mem  "maria" m)
let _ = assert (not (StringMap.mem "pedro" m))
```

Modules and Functors (module-to-module functions)

```
module type OrderedType = sig (* from the standard library *)
  type t
  val compare : t -> t -> int
end

module Pair = struct
  type t = int * string
  let compare (x1, y1) (x2, y2) =
    if x1 < x2 then -1
    else if x1 > x2 then 1
    else
    else
    else
end

module

let s =
let s =
let s =
let s = PairSet.add (3, "three") s
let s = PairSet.add (4, "four") s
let _ = assert (PairSet.mem (2, "two") s)
```

```
module PairMap = Map.Make(Pair)

let m = PairMap.empty
let m = PairMap.add (1, "a") "first"  m
let m = PairMap.add (2, "b") "second" m

let _ = assert (PairMap.find (1, "a") m = "first")
let _ = assert (not (PairMap.mem (3, "c") m))
```

Mutability

Mutability

- OCaml is not a pure language, it is possible to program side-effects.
- Examples of side-effects: I/O (printing), reads and writes of files, variables state, data bases, communication in general.
- Mutability allows the implementation of data structures more efficient than strictly pure ones (e.g., hash table, doubly linked lists, cyclic graphs).
- Mutability makes programming more difficult. It is not easy to reason about state changes (without enumerating every step).

Refs!

- A value of type `ref` can be seen as a pointer in C, or as a reference to an object or array in Java.
- The creation and dereference operations are explicit, like `malloc` or `*` in C.
- Creation and dereferencing are different from the use of state variables (stack) in imperative languages (C, Java, etc.)

```
let x = ref 0;;
```
[1] ✓ 0.0s

```
val x : int ref = {contents = 0}
```

```
!x;;
```
[2] ✓ 0.0s

```
- : int = 0
```

```
x := !x + 1
```
[3] ✓ 0.0s

```
- : unit = ()
```

```
!x;;
```
[4] ✓ 0.0s

```
- : int = 1
```

Physical equality

- Recall that the (=) operator in OCaml tests the equality of elements by their structure.
- And the function (==) tests if two references are the same. This is interesting for algorithms over data structures (references, arrays, byte sequences, records with mutable fields, etc.)

```
let r1 = ref 42
let r2 = ref 42

let _ = assert (not (r1 == r2))
let _ = assert (r1 ≠ r2)
let _ = assert (!r1 == !r2)
let _ = assert (r1 = r2)
```

[35] ✓ 0.0s

```
val r1 : int ref = {contents = 42}

val r2 : int ref = {contents = 42}

- : unit = ()

- : unit = ()

- : unit = ()

- : unit = ()
```

Aliasing

- By introducing reference, we also introduce aliasing: "Stack names allow for more than one path to the same memory cell".
- With aliasing, reasoning about the program becomes even more difficult. To analyze the independence of two code segments (threads, caller/callee) we must eliminate/control the aliasing.
- Some imperative languages, such as Rust, by design do not allow for aliasing.

```
let x = ref 42;;
let y = ref 42;;
let z = x;;
x := 43;;
let w = !y + !z;;
```

```
[7]  ✓ 0.0s
```

```
val x : int ref = {contents = 42}
val y : int ref = {contents = 42}
val z : int ref = {contents = 42}
- : unit = ()
val w : int = 85
```

Aliasing

- By intr
introd
more
cell".
- With a
progr
analy
segm
elimir
- Some
Rust,

Rust

```rust
fn main() {
    let mut x = 42;

    let r1 = &mut x;
    let r2 = &mut x; // COMPILE ERROR: cannot borrow `x`
                     // as mutable more than once at a time

    *r1 += 1;
    println!("{}", r2);
}
```

Aliasing

- By intr
introd
more
cell".
- With a
progr
analy
segm
elimir
- Some
Rust,

Rust

```rust
fn main() {
    let mut x = 42;
    let z = &x;          // shared borrow — z aliases x

    x = 43;              // COMPILE ERROR: cannot assign to `x`
                         // because it is borrowed

    let w = *z;
    println!("{}", w);
}
```

Counter example

- The function `next_val` returns a different value every time it is called. It has side-effects.
- The creation of a variable has to be separated from the function that increments it.

```
let next_val_broken = fun () →
  let counter = ref 0 in
  incr counter;
  !counter
```

[14] ✓ 0.0s

```
val next_val_broken : unit → int = <fun>
```

```
next_val_broken ();;
next_val_broken ();;
```

[15] ✓ 0.0s

```
- : int = 1

- : int = 1
```

```
let counter = ref 0

let next_val =
  fun () →
    counter := !counter + 1;
    !counter
```

[10] ✓ 0.0s

```
val counter : int ref = {contents = 0}

val next_val : unit → int = <fun>
```

```
next_val ();;

next_val ();;
```

[11] ✓ 0.0s

```
- : int = 1

- : int = 2
```

Counter example

- The function `next_val` returns a different value every time it is called. It has side-effects.
- The creation of a variable has to be separated from the function that increments it.

```
module Counter1 = Counter.Make();;
Counter1.next_val ();;
Counter1.next_val ();;
module Counter2 = Counter.Make();;
Counter2.next_val ();;
```

[28] ✓ 0.0s

```
module Counter1 :
  sig type t = int ref val counter : int ref val next_val : unit → int end

- : int = 1

- : int = 2

module Counter2 :
  sig type t = int ref val counter : int ref val next_val : unit → int end

- : int = 1
```

```
module Counter = struct
  module Make() = struct
    type t = int ref
    let counter = ref 0
    let next_val () =
      counter := !counter + 1;
      !counter
  end
end
```

[25] ✓ 0.0s

```
module Counter :
  sig
    module Make :
      functor () →
        sig
          type t = int ref
          val counter : int ref
          val next_val : unit → int
        end
  end
```